

Database Concepts

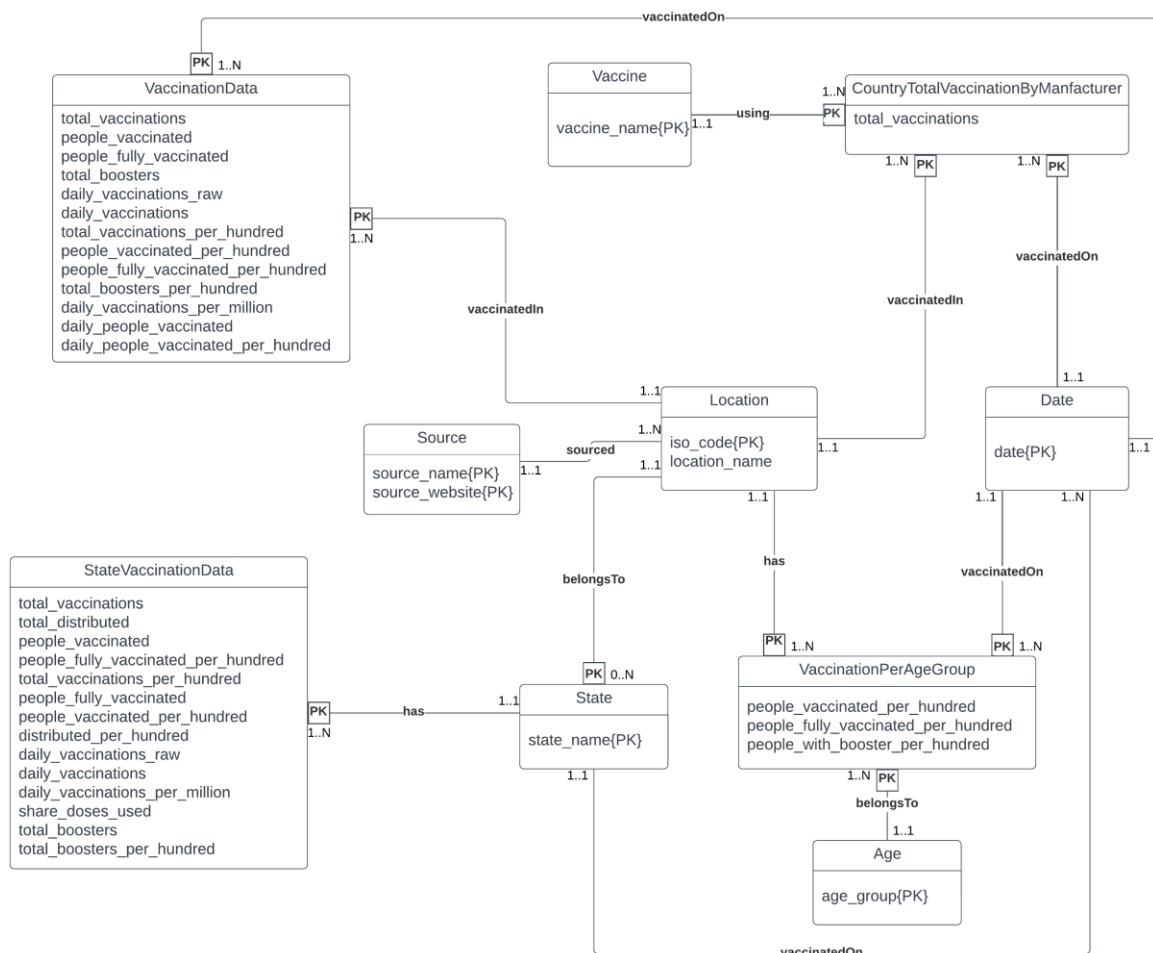
Sowmiya Kumar

Master of Data Science

Table of Contents

Database ER diagram	1
Normalization	3
Final Schema	5

Database ER diagram:



Assumptions:

For a location, the vaccination data record exists always (i.e.) it can be zero for any of the columns but not all the columns.

A location can have 0 to many states

One source_name & source_website combination serves as a source for many locations.

Strong Entities
Location
Date
Vaccine
Source
Age

Weak Entities
VaccinationData
CountryTotalVaccinationsByManufacturer
StateVaccinationData
State
VaccinationsPerAgeGroup

Based on the insights from dataset,

Attributes like Location and date were commonly present in all the CSV files. So, they are created as separate entities with their Primary key as iso_code and date.

Other simple attributes related to vaccinations don't have any unique values and they can only be identified based on the location(iso_code) and date. So they are clubbed as a weak entity called VaccinationData with location and date as their composite Foreign key. This entity has a many-to-one relationship with Location and Date.

The United States has vaccination information categorized based on state. So, they are created as a separate entity with state_name as their key. In, order to make it generic for future data, an entity called state is created and StateVaccinationData is marked as a weak entity and state_name as its Foreign key. This entity has a many-to-one relationship with Location and Date.

CountryTotalVaccinationByManufacturer and VaccinationPerAgeGroup are designed to explore the vaccinations of a country based on vaccine and age category. They are weak entities with iso_code and date as their composite Foreign key. These two entities have a many-to-one relationship with Location and Date.

Location includes countries and sub-regions within a country. Every location has a source for data. One Source name has multiple source websites and one source website has multiple source names which signifies a many-to-many relationship, so they are created as a separate entity, with source_website and source_name as a composite Primary key.

7-step process:

Step 1: Strong entities

- Location
- Date
- Vaccine
- Source
- Age

Step 2: Weak entities

- VaccinationData
- CountryTotalVaccinationsByManufacturer
- StateVaccinationData
- State
- VaccinationsPerAgeGroup

Step 3: one-to-one relationship

There is no 1:1 relationship

Step 4: one-to-many relationship

Location and source entity have a one-many relationship. So, introducing a new entity by merging the single side to the many side.

DataSource(source_name*,source_website*,iso_code*)

Step 5: many-to-many relationship

There is no N: N relationship

Step 6: Multi-valued attributes

Vaccine is a multi-valued attribute and it is separated as a single entity.

Step 7: Higher-order relationship

There is no higher order relationship

Therefore the relational schema based on the ER diagram is,

Location(iso_code, location_name)

VaccinationData(iso_code*, date*, total_vaccinations, people_vaccinated, people_fully_vaccinated, total_boosters, daily_vaccinations_raw, daily_vaccinations, total_vaccinations_per_hundred, people_vaccinated_per_hundred, people_fully_vaccinated_per_hundred, total_boosters_per_hundred, daily_vaccinations_per_million, daily_people_vaccinated, daily_people_vaccinated_per_hundred)

VaccinationsPerAgeGroup(iso_code*, date*, age_group*, people_vaccinated_per_hundred, people_fully_vaccinated_per_hundred, people_with_booster_per_hundred)

CountryTotalVaccinationByManufacturer (iso_code*, date*, vaccine_name*, total_vaccinations)

StateVaccinationData(iso_code*, date*, state*, total_vaccinations, total_distributed, people_vaccinated, people_fully_vaccinated_per_hundred, people_fully_vaccinated, people_vaccinated_per_hundred, distributed_per_hundred, daily_vaccinations_raw, daily_vaccinations, daily_vaccinations_per_million, share_doses_used, total_boosters, total_boosters_per_hundred)

Source(source_name, source_website)

Vaccine(vaccine_name)

State(state_name)

AgeCategory(age_group)

DataSource(source_name*,source_website*,iso_code*)

Normalization:

To find the candidate key(s) for the database schema, minimal set of attributes that can uniquely identify all other attributes in the database should be determined which is called closure.

Functional dependencies (FDs) for the database schema:

$\text{iso_code} \rightarrow \text{location_name}, \text{state_name}, \text{source_name}, \text{source_website}$

(iso_code, date) → total_vaccinations, people_vaccinated, people_fully_vaccinated, total_boosters, daily_vaccinations_raw, daily_vaccinations, total_vaccinations_per_hundred, people_vaccinated_per_hundred, people_fully_vaccinated_per_hundred, total_boosters_per_hundred, daily_vaccinations_per_million, daily_people_vaccinated, daily_people_vaccinated_per_hundred

(iso_code, date, age_group) → people_vaccinated_per_hundred, people_fully_vaccinated_per_hundred, people_with_booster_per_hundred

(iso_code, date, vaccine_name) → total_vaccinations

(iso_code, date, state_name) → total_vaccinations, total_distributed, people_vaccinated, people_fully_vaccinated_per_hundred, people_fully_vaccinated, people_vaccinated_per_hundred, distributed_per_hundred, daily_vaccinations_raw, daily_vaccinations, daily_vaccinations_per_million, share_doses_used, total_boosters, total_boosters_per_hundred`

source_name → source_website

(source_name, source_website) → iso_code`

vaccine_name → no functional dependencies

state_name → no functional dependencies

age_group → no functional dependencies

Closure:

iso_code+ = {iso_code, location_name, state_name, age_group, vaccine_name}

(iso_code, date)+ = {iso_code, date, total_vaccinations, people_vaccinated, people_fully_vaccinated, total_boosters, daily_vaccinations_raw, daily_vaccinations, total_vaccinations_per_hundred, people_vaccinated_per_hundred, people_fully_vaccinated_per_hundred, total_boosters_per_hundred, daily_vaccinations_per_million, daily_people_vaccinated, daily_people_vaccinated_per_hundred}

(iso_code, date, age_group)+ = {iso_code, date, age_group, state_name, people_vaccinated_per_hundred, people_fully_vaccinated_per_hundred, people_with_booster_per_hundred, location_name, total_vaccinations, people_vaccinated, people_fully_vaccinated, total_boosters, daily_vaccinations_raw, daily_vaccinations, total_vaccinations_per_hundred, people_vaccinated_per_hundred, people_fully_vaccinated_per_hundred, total_boosters_per_hundred, daily_vaccinations_per_million, daily_people_vaccinated, daily_people_vaccinated_per_hundred}

(iso_code, date, state_name, age_group, vaccine_name)+ = {iso_code, date, state_name, age_group, vaccine_name, total_vaccinations, people_vaccinated, people_fully_vaccinated, total_boosters, daily_vaccinations_raw, daily_vaccinations, total_vaccinations_per_hundred, people_vaccinated_per_hundred, people_fully_vaccinated_per_hundred, total_boosters_per_hundred, daily_vaccinations_per_million, daily_people_vaccinated, daily_people_vaccinated_per_hundred, people_vaccinated_per_hundred, people_fully_vaccinated_per_hundred, people_with_booster_per_hundred, total_distributed, distributed_per_hundred, share_doses_used, location_name, source_website}

Attributes like vaccine_name, state_name, age_group has indirect relationship with Location and Date attributes. That is, they can be indirectly derived from Location and Date. And these attributes combinely can determine all the weak entity attributes. So, based on the Functional dependencies and Closure, below is the composite Candidate key for the database

{iso_code, date}

Normalization check:

- Location, Source, DataSource, Vaccine, State, Age → 3NF

All the above mentioned entities are strong/decomposed relations.

1NF: All values are atomic.

2NF: there are no partial dependencies.

3NF: all the attributes are functionally dependent on candidate key and there is no transitive dependencies.

- VaccinationData, VaccinationsPerAgeGroup, CountryTotalVaccinationByManufacturer, StateVaccinationData

All the above entities are weak entities. They have composition relation with the strong entities.

1NF: All values are atomic.

2NF: All non-key attributes are functionally dependent on the candidate key.

3NF: all the attributes are functionally dependent on candidate key and there is no transitive dependencies.

The Final Schema:

Vaccine (vaccine_name)

Location(iso_code, location_name)

Source (source_name, source_website)

DataSource(source_name*, source_website*, iso_code*)

Date (date)

Age(age_group)

VaccinationPerAgeGroup (iso_code*, date*, age_group, people_vaccinated_per_hundred, people_fully_vaccinated_per_hundred, people_with_booster_per_hundred)

VaccinationData(iso_code*, date*, total_vaccinations, people_vaccinated, people_fully_vaccinated, total_boosters, daily_vaccinations_raw, daily_vaccinations, total_vaccinations_per_hundred, people_vaccinated_per_hundred, people_fully_vaccinated_per_hundred, total_boosters_per_hundred, daily_vaccinations_per_million, daily_people_vaccinated, daily_people_vaccinated_per_hundred)

CountryTotalVaccinationByManufacturer (iso_code*, date*, vaccine_name*, total_vaccinations)

StateVaccinationData(iso_code*, state_name*, date*, total_vaccinations, total_distributed, people_vaccinated, people_fully_vaccinated_per_hundred, people_fully_vaccinated, people_vaccinated_per_hundred, distributed_per_hundred, daily_vaccinations_raw, daily_vaccinations, daily_vaccinations_per_million, share_doses_used, total_boosters, total_boosters_per_hundred)

State(iso_code*, state_name)